Course Name: Software Metrics

Course Code: ISWE402L
Slot: B2+ TB2

Review-1

Project Title

Developer Productivity Metrics Dashboard

Reg. No.	Name
23MIS0576	H.PRAJWAL
23MIS0692	VR.SHARUKESHWARAN

1. Project Overview

The **Developer Productivity Metrics Dashboard** is an analytics-based software project designed to measure and visualize the productivity and working patterns of software developers. The system focuses on understanding developer productivity through multiple dimensions rather than relying only on simple activity measures such as the number of commits.

In a software development environment, developers continuously generate data through pull requests, code reviews, issues, task updates, commits, and other collaborative activities. This project collects such development activity data and transforms it into meaningful productivity metrics.

The dashboard analyses aspects such as development throughput, review responsiveness, task completion, workload distribution, collaboration, inactive work, change hotspots, delivery consistency, and developer focus.

The system will use **GitHub** as the primary online development platform and data source. Information such as pull requests, issues, reviews, commits, timestamps, changed files, and developer interactions will be collected using the GitHub REST API.

The collected information will be processed and converted into **12 productivity metrics**. Ten metrics will provide activity and productivity measurements, while two metrics — **Focus Stability Score (FSS)** and **Productivity Recovery Index (PRI)** — will be proposed as custom metrics specifically for this project.

The main goal is to provide an interactive dashboard that helps developers and project managers understand productivity patterns, identify bottlenecks, analyse workload distribution, and improve the efficiency of software development processes.

2. Objectives

➢ Develop an interactive dashboard for analysing developer productivity.

➢ Collect real software development data from GitHub.

➢ Analyse pull requests, reviews, issues, tasks, commits, and developer interactions.

➢ Measure developer productivity using multiple dimensions instead of a single activity measure.

➢ Identify delays in pull request reviews and development workflows.

➢ Analyse workload distribution among developers.

➢ Identify inactive or stale development work.

➢ Measure collaboration patterns between developers.

➢ Identify frequently modified code areas and development hotspots.

➢ Measure consistency between planned and completed development work.

➢ Introduce two custom metrics for measuring developer focus and recovery from interruptions.

➢ Provide visual insights that can help teams identify productivity bottlenecks and improve development processes.

3. System Architecture

The proposed system follows a **five-stage analytics architecture**.

1. Data Source Layer

This layer contains the online development platform from which project data is collected.

Primary Source:

GitHub

The system collects:

• Pull Requests
• Pull Request Reviews
• Issues
• Commits
• Changed Files
• Developers
• Labels
• Timestamps
• Merge Information
• Issue Status

2. Data Collection Layer

This layer communicates with GitHub using the **GitHub REST API**.

The data collection module retrieves:

• Developer activity
• Pull request information
• Review activity
• Issue creation and closure
• File modification information
• Development timestamps
• Merge events

The collected data is stored in structured JSON, CSV, or database format for further processing.

3. Data Processing Layer

The raw development information is cleaned and transformed before metric calculation.

Major processing activities include:

• Removing duplicate records
• Handling missing values
• Converting timestamps
• Grouping activities by developer
• Grouping activities by project period
• Identifying inactive work
• Identifying repeated changes
• Calculating development durations
• Identifying collaboration relationships

4. Metrics Engine

The Metrics Engine calculates the selected productivity metrics using predefined formulas.

It contains:

• Pull Request Analysis
• Review Analysis
• Task Analysis
• Workload Analysis
• Collaboration Analysis
• Change Analysis
• Productivity Analysis
• Custom Metric Calculation

The Metrics Engine produces normalized values that can be displayed in the dashboard.

5. Dashboard Layer

The final layer presents the calculated metrics through an interactive dashboard.

Dashboard Views:

• Overall Productivity Overview
• Developer Comparison
• Pull Request Analytics
• Review Performance
• Task Completion
• Workload Distribution
• Collaboration Network
• Change Hotspots
• Productivity Trends
• Custom Metric Analysis

The dashboard allows users to filter results by developer, date range, repository, issue type, or development period.

4. Expected Outcome

The expected outcome of the project is a functional **Developer Productivity Metrics Dashboard** capable of collecting development data, calculating productivity metrics, and presenting the results through interactive visualizations.

The system will provide:

➢ Developer-wise productivity analysis.

➢ Pull request and review performance analysis.

➢ Task completion and workload analysis.

➢ Identification of stale development work.

➢ Collaboration pattern visualization.

➢ Identification of frequently changed development areas.

➢ Delivery consistency analysis.

➢ Custom productivity measurements.

➢ Time-based productivity trends.

➢ Data-driven insights for improving software development efficiency.

The final dashboard will demonstrate how software development activity can be converted into meaningful quantitative productivity information.

5. Technology Stack

• **Programming Language:** Python

• **Dashboard Framework:** Streamlit

• **Data Analysis:** Pandas

• **Data Visualization:** Plotly

• **Online Data Source:** GitHub

• **API:** GitHub REST API

• **Storage:** CSV / JSON / SQLite

• **Version Control:** Git & GitHub

6. Methodology

The project begins by identifying a GitHub repository containing software development activity. The GitHub REST API will be used to collect development information such as pull requests, reviews, issues, commits, changed files, developers, and timestamps.

The collected data will be stored in a structured format and passed through a preprocessing stage. During preprocessing, missing values, duplicate records, timestamps, developer identities, and activity categories will be handled.

The processed data will then be passed to the **Metrics Engine**. Each metric will be calculated according to its corresponding formula. Time-based metrics will use timestamps obtained from GitHub, while developer and collaboration metrics will be calculated by grouping activities according to users and project components.

The two custom metrics, **Focus Stability Score (FSS)** and **Productivity Recovery Index (PRI)**, will be calculated using development activity and interruption-related information.

Finally, the calculated metrics will be displayed through a Streamlit dashboard. Charts, tables, KPI cards, rankings, and trend graphs will be used to make the results easy to understand.

The results will be analysed to identify productivity bottlenecks, workload imbalance, review delays, inactive work, development hotspots, and productivity fluctuations.

7. Software Metrics Applied

The following **12 metrics** will be applied to the project. The metrics focus specifically on developer productivity, development workflow, collaboration, workload, and consistency.

Development Metrics

No.	Metric	Description
1	Pull Request Throughput (PRT)	Measures the number of successfully merged pull requests completed during a specific period.
2	Pull Request Lead Time (PRLT)	Measures the time taken from pull request creation until it is successfully merged.
3	Review Responsiveness Time (RRT)	Measures how quickly developers or reviewers respond after a pull request is submitted for review.
4	Pull Request Iteration Count (PRIC)	Measures the number of modification and review cycles required before a pull request is merged.
5	Stale Work Ratio (SWR)	Measures the percentage of issues or pull requests that remain inactive beyond a defined period.
6	Task Completion Efficiency (TCE)	Measures the relationship between completed development tasks and the time required to complete them.
7	Developer Workload Balance (DWB)	Measures how evenly development work is distributed among team members.
8	Collaboration Reach Score (CRS)	Measures the breadth of collaboration between a developer and other members of the development team.
9	Change Hotspot Frequency (CHF)	Measures how frequently particular files or modules are modified within a given period.
10	Delivery Predictability Rate (DPR)	Measures how consistently planned development work is completed within its expected time period.
11	Focus Stability Score (FSS) – Custom Metric	Measures the consistency of a developer's focused development activity by considering task switching and uninterrupted work sessions.
12	Productivity Recovery Index (PRI) – Custom Metric	Measures how effectively and quickly a developer returns to productive work after development interruptions.

8. Detailed Metric Description

1. Pull Request Throughput — PRT

Pull Request Throughput measures the number of pull requests successfully merged within a particular period.

Formula:

PRT = Number of Merged Pull Requests / Time Period

For example, if 20 pull requests are merged during one month:

PRT = 20 PRs/month

This metric provides an indication of the team's development delivery volume.

2. Pull Request Lead Time — PRLT

Pull Request Lead Time measures how long a pull request takes to move from creation to successful merging.

Formula:

PRLT = Merge Time − Pull Request Creation Time

A lower lead time generally indicates faster development and integration.

The dashboard can display:

• Average PR Lead Time
• Median PR Lead Time
• Developer-wise PR Lead Time

3. Review Responsiveness Time — RRT

Review Responsiveness Time measures the time between submitting a pull request and receiving the first review response.

Formula:

RRT = First Review Time − PR Submission Time

A high value may indicate reviewer availability problems or a review bottleneck.

4. Pull Request Iteration Count — PRIC

This metric measures how many development-review cycles occur before a pull request is finally merged.

Example:

PR submitted → Review → Changes → Review → Changes → Merge

Formula:

PRIC = Total Review/Revision Cycles / Merged Pull Requests

Lower values generally indicate smoother development and review processes.

5. Stale Work Ratio — SWR

Stale Work Ratio identifies work items that remain inactive for longer than a predefined threshold.

Examples:

• Unupdated pull requests
• Unresolved issues
• Blocked development tasks

Formula:

SWR = Number of Stale Work Items / Total Active Work Items × 100

A high SWR indicates that a significant portion of project work is becoming inactive.

6. Task Completion Efficiency — TCE

Task Completion Efficiency measures the relationship between successfully completed tasks and the time required to complete them.

Formula:

TCE = Completed Tasks / Total Development Time

Higher TCE indicates that more work is being completed within the available development time.

7. Developer Workload Balance — DWB

Developer Workload Balance measures how evenly development work is distributed across team members.

Formula:

Developer Workload % = Developer Workload / Total Team Workload × 100

The metric can identify:

• Overloaded developers
• Underutilized developers
• Uneven workload distribution

8. Collaboration Reach Score — CRS

Collaboration Reach Score measures how broadly a developer interacts with other developers through pull requests, reviews, issues, and shared development activities.

Formula:

CRS = Number of Unique Collaborators / Total Team Members × 100

Higher CRS indicates broader collaboration.

9. Change Hotspot Frequency — CHF

Change Hotspot Frequency identifies files or modules that are modified repeatedly.

Formula:

CHF = Number of Changes to Component / Analysis Period

Frequently modified components may indicate:

• High development activity
• Unstable requirements
• Complex functionality
• Maintenance-heavy modules

10. Delivery Predictability Rate — DPR

Delivery Predictability Rate measures how consistently planned development work is completed within the expected time.

Formula:

DPR = Work Items Completed Within Expected Time / Total Completed Work Items × 100

Higher DPR indicates more predictable development delivery.

11. Focus Stability Score — FSS

Custom Metric

The **Focus Stability Score** is a proposed metric developed specifically for this project. It measures how consistently developers remain focused on active development work.

The metric considers:

• Development sessions
• Task switching
• Interrupted work
• Completed work sessions

Proposed Formula:

FSS = 100 × (Focused Development Sessions / Total Development Sessions)

The score ranges from **0 to 100**.

Score	Interpretation
80–100	Highly stable focus
60–79	Good focus
40–59	Moderate interruption
Below 40	High task switching / interruption

12. Productivity Recovery Index — PRI

Custom Metric

The **Productivity Recovery Index** is the second proposed metric of the project.

Software developers can experience interruptions such as:

• Failed builds
• Blocked pull requests
• Review delays
• Reopened issues
• Development dependencies
• Interrupted tasks

PRI measures how effectively developers return to productive work after such interruptions.

Proposed Formula:

PRI = Productive Recovery Events / Total Disruption Events × 100

Score	Interpretation
80–100	Excellent recovery
60–79	Good recovery
40–59	Moderate recovery
Below 40	Poor recovery

Higher PRI indicates better recovery from development disruptions.

9. Online Tool Used for the Project

GitHub

GitHub will be used as the primary online development platform and data source for the project.

GitHub is suitable because software development activities are continuously recorded within repositories. These activities can be accessed through the GitHub API and transformed into productivity metrics.

Data collected from GitHub:

➢ Pull Requests
➢ Pull Request Reviews
➢ Issues
➢ Commits
➢ Developers
➢ Changed Files
➢ Labels
➢ Timestamps
➢ Merge Events
➢ Issue Status

The collected information will be processed and used as input for the productivity metrics engine.

10. Role of GitHub in the Project

GitHub will act as the primary source of real development activity data.

The system will use GitHub data for:

➢ Measuring pull request throughput.

➢ Calculating pull request lead time.

➢ Measuring review responsiveness.

➢ Identifying review and revision cycles.

➢ Detecting stale issues and pull requests.

➢ Analysing developer workload.

➢ Measuring collaboration between developers.

➢ Identifying frequently modified files.

➢ Analysing development delivery patterns.

➢ Supporting the calculation of custom productivity metrics.

The GitHub API will provide the required development events, while the dashboard will convert these events into meaningful productivity information.

11. Dashboard Design

The proposed dashboard will contain multiple sections.

Overall Productivity

Displays:

• Total Developers
• Total Pull Requests
• Total Issues
• Average PR Lead Time
• Average Review Time
• Team Productivity Indicators

Developer Comparison

Displays:

• Developer-wise PRT
• TCE
• DWB
• CRS
• FSS
• PRI

This allows comparison of different productivity dimensions without reducing productivity to one number.

Development Workflow

Displays:

• Pull Request Throughput
• PR Lead Time
• Review Responsiveness
• PR Iteration Count

Charts can show how the development workflow changes over time.

Workload Analysis

Displays:

• Developer workload percentage
• Workload distribution
• Active work
• Stale work
• Task completion

Development Hotspots

Displays:

• Frequently modified files
• Frequently changed modules
• Number of changes
• Change trends

Custom Metrics

Displays:

Focus Stability Score — FSS

Productivity Recovery Index — PRI

Trend graphs can show whether these values improve or decline over time.

12. Project Workflow

GitHub Repository

↓

GitHub REST API

↓

Development Data Collection

↓

Data Cleaning & Processing

↓

Metric Calculation Engine

↓

Standard Metrics + Custom Metrics

↓

Streamlit Dashboard

↓

Visualization & Developer Productivity Analysis

↓

Insights & Recommendations

13. Abstract

The **Developer Productivity Metrics Dashboard** is an analytics-based software metrics project developed to measure, analyse, and visualize developer productivity using real software development activity. Conventional productivity measurements often focus on simple activity indicators such as commit counts, which may not adequately represent the efficiency and effectiveness of software development. This project therefore adopts a multi-dimensional approach that considers development throughput, review responsiveness, task completion, workload distribution, collaboration, development hotspots, delivery consistency, developer focus, and recovery from interruptions.

The proposed system uses **GitHub** as the primary online data source. Development information such as pull requests, reviews, issues, commits, changed files, developers, timestamps, and merge events will be collected using the GitHub REST API. The collected data will be cleaned, processed, and transformed into meaningful productivity metrics.

A total of **12 metrics** will be applied to the project. These include Pull Request Throughput, Pull Request Lead Time, Review Responsiveness Time, Pull Request Iteration Count, Stale Work Ratio, Task Completion Efficiency, Developer Workload Balance, Collaboration Reach Score, Change Hotspot Frequency, and Delivery Predictability Rate. Two additional metrics, **Focus Stability Score (FSS)** and **Productivity Recovery Index (PRI)**, are proposed as custom metrics specifically for this project.

FSS measures the consistency of focused development activity by considering development sessions and task switching, while PRI measures how effectively developers return to productive work after interruptions such as review delays, failed builds, blocked tasks, or reopened issues.

The calculated metrics will be displayed through an interactive dashboard developed using **Python, Pandas, Streamlit, and Plotly**. The dashboard will provide developer comparisons, productivity trends, workload distribution, development workflow analysis, collaboration information, change hotspots, and custom metric visualizations.

The final objective of the project is to demonstrate that developer productivity can be analysed more effectively by considering multiple dimensions of software development rather than relying on a single activity measure. The dashboard will help identify productivity bottlenecks, workload imbalance, review delays, inactive work, collaboration patterns, and development interruptions, thereby supporting data-driven decision-making for software development teams.